OOAD Principles

A practical interview quick reference for object-oriented analysis and design

The goal is not to memorize 100 designs.
Build a repeatable way to discover domain objects, assign responsibilities, model behavior, and protect correctness.

Use this guide while practising problems such as Meeting Room Scheduler, Ride Sharing, ATM, Order Management, Rate Limiter, API Gateway, and Task Scheduler.

Core loop: **Requirements** → **Objects** → **Responsibilities** → **Workflows** → **Abstractions** → **Validation**

1. The reusable OOAD workflow

Start with requirements and let the design emerge. Do not start by choosing design patterns.

Step	Ask yourself	What to produce
1. Understand requirements	What are the main use cases, constraints, edge cases, and non-functional needs?	Scope and use-case list
2. Identify domain objects	What concepts have identity, data, lifecycle, or business meaning?	Entities, value objects, relationships
3. Assign responsibilities	Who owns this data? Who protects the rule? Who coordinates the operation?	Cohesive methods and interfaces
4. Model workflows and state	What happens first, next, on failure, and on cancellation?	Sequence and state transitions
5. Introduce abstractions	What varies independently? What external interface must be adapted?	Interfaces and selective patterns
6. Validate the design	What if two requests race, a dependency fails, or the task is retried?	Invariants, failure handling, tests

Interview habit: state assumptions explicitly. Design for the requirements given, and add complexity only when a requirement justifies it.

Quick requirement checklist

- Functional: actors, primary actions, alternate paths, cancellation and failure.
- Domain rules: what must always be true?
- Scale and performance: throughput, latency, expected contention.
- Reliability: persistence, retries, recovery, idempotency.
- Concurrency: can two users claim the same resource at once?

2. Objects, relationships, and ownership

Relationship	Meaning	Typical signal
Association	Objects collaborate or know about each other; lifecycles remain independent.	Meeting ↔ User; Ride ↔ Driver
Aggregation	A whole refers to parts that can exist independently; often optional in practical UML.	A team references existing employees
Composition	The whole strongly owns a part's lifecycle.	Order → OrderLine, when lines have no independent business lifecycle
Inheritance	An is-a relationship with a valid substitutable subtype.	Car extends Vehicle, if the subtype honors the Vehicle contract

Do not confuse containment with ownership. A reference or collection does not automatically mean composition.

Multiplicity and relationship objects

- State cardinality where useful: one-to-one, one-to-many, many-to-many.
- If a relationship has its own attributes, identity, lifecycle, or rules, model it as an object.
- Example: Student ↔ Course often becomes Student → Enrollment → Course when enrollment has grade, date, or status.
- Choose ownership by asking: can the part exist independently, and who controls its creation, deletion, and invariants?

3. Where should behavior live?

Component	Primary responsibility	Example
Domain object / entity	Protect its state and invariants; implement behavior naturally belonging to it.	order.cancel(); booking.confirm();
Application service	Coordinate a use case across objects, repositories, and external systems.	checkoutService.checkout(orderId)
Domain service	Represent business logic that does not naturally belong to one entity/value object.	A policy comparing several domain concepts
Repository	Load and persist domain objects; hide storage details.	orderRepository.findById(id)
Adapter / gateway	Translate the system's interface to an external API or technology.	paymentGateway.charge(...)

Three-question decision

- **Whose state or invariant is protected?** Start with that domain object.
- **Does it coordinate multiple objects or systems?** Put the workflow in an application service.
- **Is it storage or external integration?** Use a repository or adapter/gateway.

Prefer booking.cancel() over booking.setStatus(CANCELLED): meaningful operations can enforce rules; unrestricted setters make invalid states easier to create.

4. Lifecycle and state transitions

Lifecycle is an object's journey. State is its current condition. A transition changes state in response to an operation or event.

Example RoomBooking: PENDING → CONFIRMED → COMPLETED; alternatively PENDING/CONFIRMED → CANCELLED. Real systems may also need EXPIRED or REJECTED.

For every important object, ask

- How is it created, and when is it terminal?
- What meaningful states can it have?
- Which events or methods cause transitions?
- Which transitions are forbidden?
- What happens if two transitions race or an operation is retried?

Pattern / technique	Use when
Enum + validated methods	Few states and straightforward transition rules
State pattern	Many states have substantially different behavior
Strategy pattern	An algorithm or policy varies independently of state
Observer / event publishing	Other components need notification of state changes

5. Pluggable algorithms and policies

If behavior is likely to vary independently, define a stable interface and provide interchangeable implementations. This is the central idea of the Strategy pattern.

Problem	Potential strategy
Ride Sharing	Economy, premium, or surge fare calculation

Problem	Potential strategy
Meeting Room Scheduler	Allocation / room-selection policy
Rate Limiter	Fixed window, sliding window, token bucket
E-commerce	Discount or shipping policy
Routing	Fastest, cheapest, or toll-avoiding route

Algorithm vs. policy

- **Algorithm:** how a calculation or operation is performed.
- **Policy:** the rule or criterion governing a decision.
- Keep simple, stable conditions as ordinary code. Introduce a strategy when variation is real, independently testable, configurable, or expected to grow.

Strategy vs. State vs. Factory

Pattern	Question answered
Strategy	Which algorithm/policy should perform this operation?
State	How does behavior depend on the object's current state?
Factory	Which object/implementation should be created?
Adapter	How do I work with an incompatible external interface?
Decorator	How can I add behavior without changing the original implementation?
Observer / Pub-Sub	How should interested components learn that something happened?

6. Resource allocation and concurrency

Begin by identifying the scarce resource and the invariant that must remain true, even when requests run simultaneously.

System	Resource	Invariant
Meeting room	Room + time interval	No overlapping confirmed bookings for the same room
Ride sharing	Driver / vehicle	No two incompatible active rides
E-commerce	Inventory quantity	Reserved/sold quantity does not exceed available stock
ATM	Cash	Dispensed amount does not exceed available cash
Rate limiter	Quota	Requests stay within the configured limit

Choose protection at the correct layer

- **In-process lock:** protects shared memory in one process; not sufficient across multiple application instances.
- **Atomic database update:** useful for simple counters and stock reservations.
- **Optimistic locking:** detects concurrent modifications; handle conflicts/retries.
- **Pessimistic locking:** locks rows/resources when coordinating multi-step database work; consider contention and deadlocks.
- **Constraints / isolation:** use database guarantees appropriate to the invariant. A check followed by an insert is not automatically race-safe.

Domain objects express business invariants; the database or distributed coordination mechanism may be required to enforce them across all competing requests.

7. Queues, scheduling, and retries

Concept	Question	Responsibility
Queue	What work is waiting?	Buffer and deliver pending tasks to consumers
Scheduler	When is work eligible?	Track execution times, recurring work, and delays
Retry policy	What happens after failure?	Classify retryable failures, delay, and attempt limits
Worker / handler	How is work done?	Execute task-specific behavior
Dead-letter queue	What exhausted retries or repeatedly failed?	Retain failed tasks for investigation/replay

Typical task lifecycle

READY → PROCESSING → COMPLETED. On retryable failure: PROCESSING → RETRY_WAIT → READY. After retry exhaustion: FAILED or DEAD_LETTER.

Reliability checklist

- Use durable storage/queues if work must survive restarts.
- Make side effects idempotent where duplicate execution is possible.
- Retry transient failures, not permanent validation failures.
- Use bounded exponential backoff, often with jitter, to avoid synchronized retry storms.
- Persist attempt count and next eligible execution time when required.
- Account for worker crashes, duplicate delivery, task claiming, cancellation, and recovery.

Queue, scheduler, and retry are distinct responsibilities, but need not always be separate classes or infrastructure services.

8. Pattern selection: start with the problem

Design signal	Pattern to consider
Interchangeable algorithm or business policy	Strategy
Behavior varies by lifecycle state	State
Object creation varies or is complex	Factory
Incompatible third-party API	Adapter
Add optional behavior around an object	Decorator
Notify subscribers about an event	Observer / Pub-Sub

- Patterns are tools, not mandatory steps. Do not introduce one merely because an if/else exists.
- Prefer composition over inheritance unless there is a genuine is-a relationship and substitutability holds.
- Depend on abstractions where variation or external boundaries justify them; avoid interfaces with no clear purpose.

9. LLD vs. HLD and final validation

Low-Level Design (LLD / OOAD)	High-Level Design (HLD)
Classes, methods, interfaces, object relationships	Services, systems, deployment boundaries
State transitions and domain invariants	Capacity, availability, scalability, data flow
Patterns, encapsulation, testability	Databases, brokers, caching, network topology
Locking / transaction behavior at operation level	Partitioning, replication, distributed coordination

Before presenting your design, check

- Are responsibilities cohesive and data encapsulated?
- Are object relationships and lifecycles justified?
- Are invalid state transitions prevented?
- Are concurrent operations safe?
- What happens on timeout, partial failure, duplicate request, and retry?
- Can behavior that truly varies be replaced without modifying its caller?
- Are assumptions and trade-offs stated explicitly?

10. A reusable interview answer template

1. **Requirements:** List core use cases, constraints, and edge cases.
2. **Domain model:** Identify entities/value objects, relationships, cardinalities, and ownership.
3. **Responsibilities:** Assign invariant-protecting behavior to domain objects; coordinate use cases in services.
4. **Workflow:** Explain the happy path, state transitions, failure paths, and cancellation.
5. **Abstractions:** Introduce interfaces/patterns only for meaningful variation or boundaries.
6. **Correctness:** Cover concurrency, transactions, idempotency, retries, and recovery where relevant.
7. **Trade-offs:** Explain why this is the simplest design that meets the stated requirements.

One-line summary: Requirements → Domain model → Responsibility assignment → Lifecycle/workflow → Selective abstractions → Concurrency and failure validation.